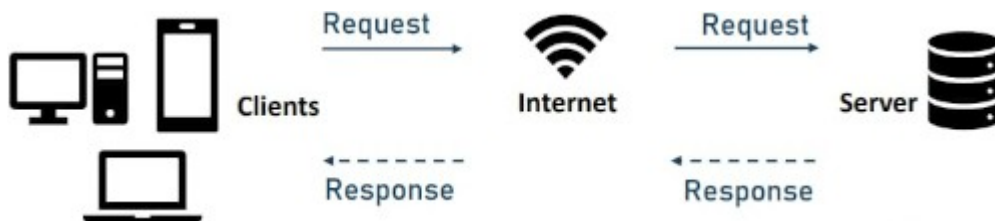


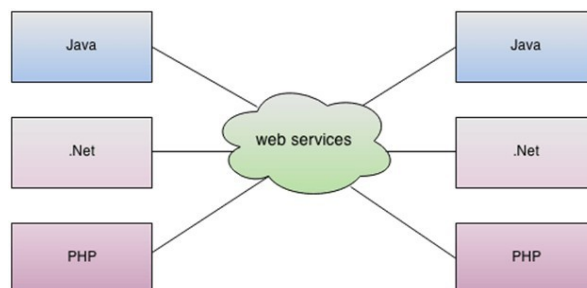
2 Introduction to Web Services

2.1 What is a web service?

- A way for two different applications – potentially written in different programming languages and running on different platforms – to communicate and exchange data using standardized web protocols like HTTP

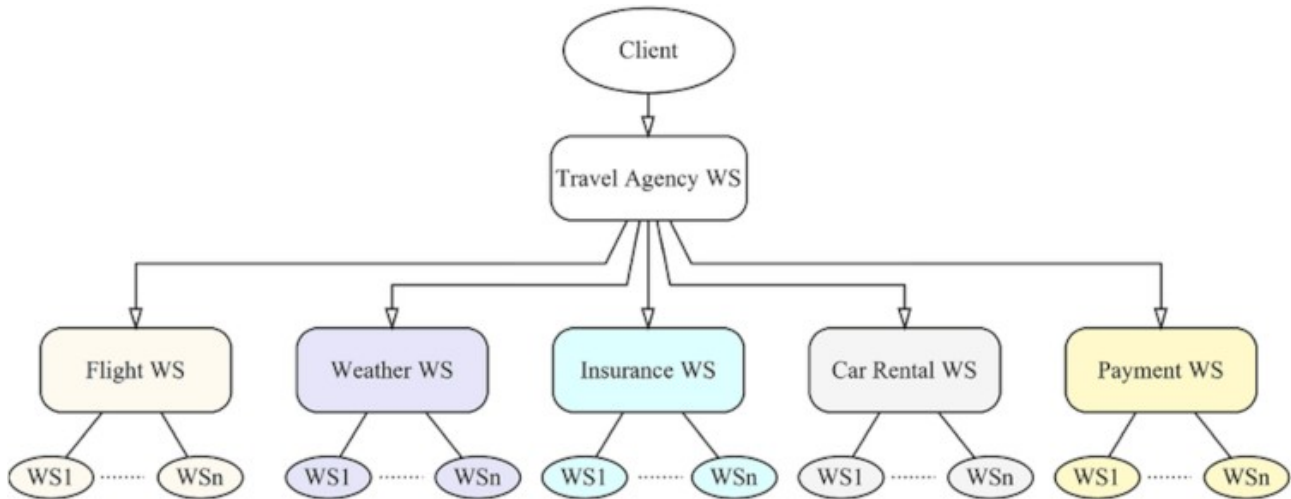


- A standards-based, language-agnostic software entity, that accepts specially formatted requests from other software entities on remote machines via vendor and transport neutral communication protocols, producing application specific responses
 - Key points
 - Standards based
 - Vendor neutral
 - Language agnostic
 - Transport neutral
 - Formatted requests
 - Application specific responses
 - Remote machines
 - Simple illustration



- As can be seen in the example
 - Java, .Net, and PHP applications can communicate with other applications through web service over the network
 - i.e, a web service is a language independent way of communication
- A means by which computers talk to each other over the web using HTTP and other universally supported protocols.

- An application that:
 - Runs on a Web server
 - Exposes Web methods to interested callers
 - Listens for HTTP requests representing commands to invoke Web methods
 - Executes Web methods and returns the results
- Example



- Web service vs API
 - All web services are APIs, but not all APIs are web services
 - Network requirement
 - A Web Service must operate over a network (like the internet or an intranet)
 - An API can be local (e.g., a library within your operating system) and does not necessarily require a network connection to function

2.2 Core functionality

- Interoperability
 - Acts as a bridge, allowing an application in one language (say Java) running on a given platform (say, a Linux server) to talk to an application in a different language (say, .NET) on a different platform (say, a Windows machine).
- Standardized Messaging
 - Data is typically exchanged using standard formats like XML or JSON
- Loose Coupling

- Client and server are independent
- A change in the server's internal code doesn't necessarily require a change in the client, as long as the interface remains the same.

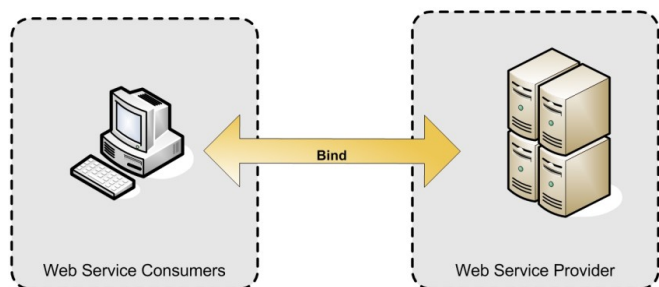
2.3 Benefits of web services

- Loose coupling, as indicated above
- Ease of Integration
 - Data is isolated between applications creating 'silos'
 - Web Services act as glue between these and enable easier communications within and across organisations.
- Service Reuse
 - Takes code reuse a step further
 - A specific function within the domain is only ever coded once and used over and over again by consuming applications

2.4 Architecture of Web Services

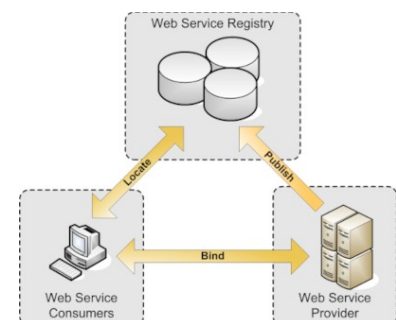
2.4.1 Simplest Model

- Just 2 components
 - A service producer (provider)
 - Presents the interface and implementation of the service
 - A service consumer (requester)
 - Uses the web service

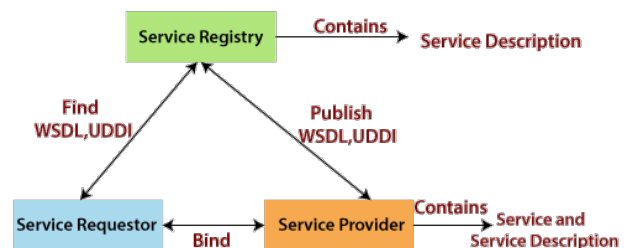
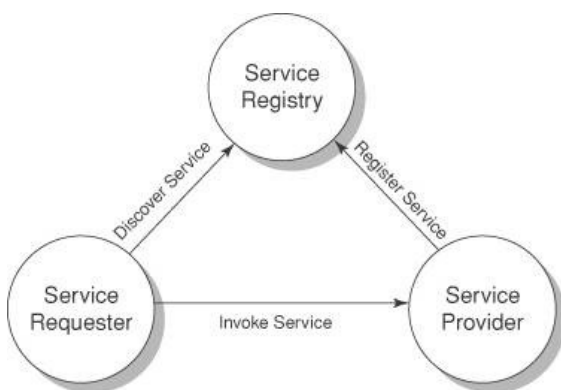


2.4.2 Service Oriented Architecture (SOA)

- Architectural style used to design software as a collection of independent, reusable services
- Focus is on business functionality rather than technical implementation
- Forms the standard infrastructure solution for building distributed applications as services that can be published, discovered, and accessed over the Internet
- Centred around standards and standards-based technologies: Internet protocols and XML
 - This ensures that the implementations of Web services applications are compliant to standard specifications, thus enabling interoperability with those compliant applications



- Web services are the standard technology used to implement and realize this architecture
- Core Components
 - Service Provider
 - Responsible for developing and deploying the Web services
 - Defines the services and publishes their interfaces to a registry
 - Service Broker/Registry
 - A directory where providers list services and consumers discover them (often using UDDI)
 - Responsible for service registration and discovery of the Web services
 - Lists the various service types, descriptions, and locations of the services that help the service requesters find and subscribe to the required services
 - Service Requester/Consumer
 - Responsible for the service invocation
 - Finds services in a registry using the service broker and binds to the provider to use them
 - Invokes the required services



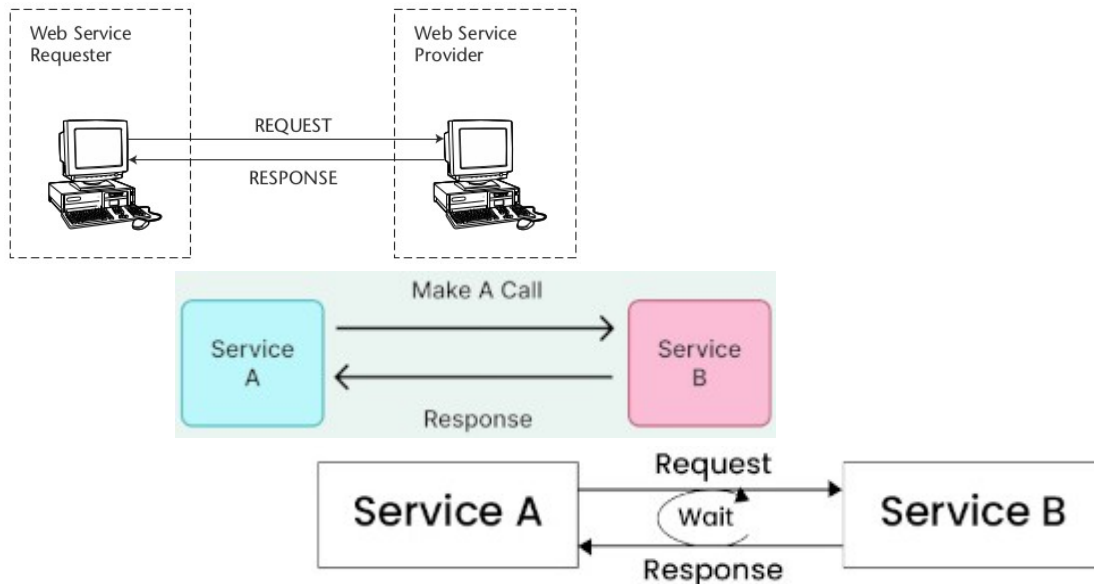
Web Service Roles, Operations and Artifacts

- Web service vs SOA
 - Concept vs. Implementation
 - SOA is the "what" (the architectural blueprint)
 - Web services are the "how" (the technical implementation)
 - Web services allow for the automatic discovery and invocation of business functions over the internet, a core goal of SOA

2.4.3 Web Services Communication Models

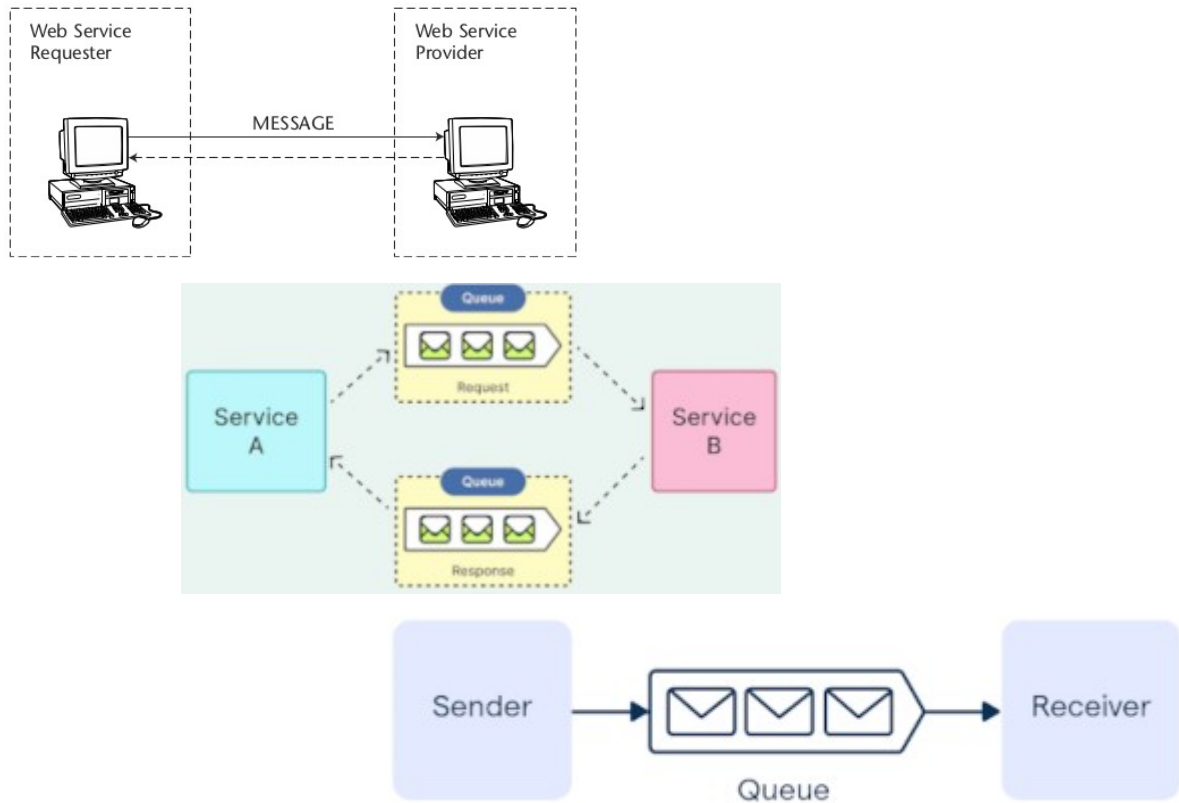
- Two communication models for web services:
 - RPC-based synchronous, or
 - Messaging-based synchronous/asynchronous communication

2.4.3.1 *RPC-Based Communication Model*



- Defines a request/response-based synchronous communication
 - When the client sends a request, the client waits until a response is sent back from the server before continuing any operation
 - Examples: CORBA, RMI communication
 - RPC-based Web services are tightly coupled and are implemented with remote objects to the client application
 - The clients have the capability to provide parameters in method calls to the Web service provider
 - Clients invoke the Web services by sending parameter values to the Web service provider that executes the required methods, and then sends back the return values.

2.4.3.2 *Messaging-Based Communication Model*



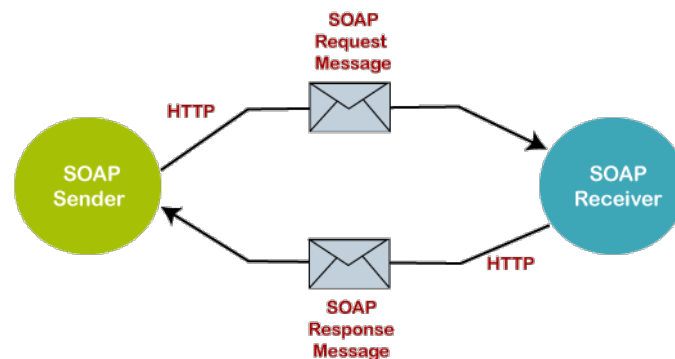
- A loosely coupled and document-driven communication model
 - The service requester invoking a messaging-based service provider does not wait for a response.
 - Client service typically sends an entire document rather than sending a set of parameters
 - The service provider receives the document, processes it, and then may or may not return a message

2.4.4 Web Services Standards

2.4.4.1 Primary Standards

- SOAP (Simple Object Access Protocol)
 - A messaging protocol for exchanging structured information
 - Uses XML as the message format, plus a set of encoding rules for representing data as messages
 - An industry accepted W3C specification for a ubiquitous XML distributed computing infrastructure
 - An application layer protocol that relies on other application layer protocols such as HTTP, SMTP, FTP (most commonly HTTP) to transport it

- Enables exchange of information between two or more peers and enables them to communicate with each other in a decentralized, distributed application environment
- Independent of the application object model, language, and running platforms or devices

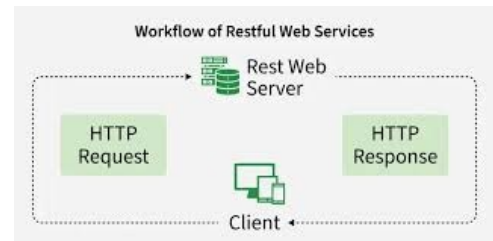
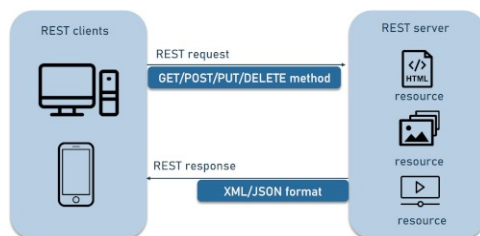


- WSDL (Web Services Description Language)
 - An XML-based language that defines the interface, methods, and endpoint of a SOAP service
 - Used as the metadata language for defining Web services and describes how service providers and requesters communicate with one another
 - A WSDL file
 - Defines a binding mechanism used to attach a protocol, data format, an abstract message, or set of endpoints defining the location of services
 - Contains information about web services such as method name, method parameter and how to access it
 - Describes the web services functionalities offered by the service provider, where the service is located, and how to access the service
- UDDI (Universal Description, Discovery, and Integration)
 - An XML-based framework for describing, discovering and integrating web services
 - Defines a mechanism to register and categorize Web services in a general-purpose registry that users communicate to in order to discover and locate registered services
 - Querying a UDDI registry for a service returns the WSDL description describing the corresponding service interface
 - Using the WSDL description, the developer can construct a SOAP client interface that can communicate with the service provider
 - Web services brokers use UDDI as a standard for registering the Web service providers

- By communicating with the UDDI registries, the service requesters locate services and then invoke them

- REST (Representational State Transfer)

- An architectural style
- A lightweight alternative to SOAP that uses standard HTTP methods (GET, POST, PUT, DELETE) and commonly JSON or XML for faster, lightweight, stateless communication



- XML (eXtensible Markup Language)

- A W3C recommendation for creating special-purpose markup languages that enable the structuring, description and interchange of data
- A simplified subset of SGML capable of describing many different kinds of data for any imaginable application domain.
- Facilitates the sharing of structured text and information in databases and across the Internet
- Languages based on XML are themselves described in a formal way, allowing programs to modify and validate documents in these languages without prior knowledge of their form

- JSON (JavaScript Object Notation)

- A lightweight alternative to XML
- Like XML
 - Text-based, language-independent data-interchange format used for storing and transporting data, primarily between servers and web applications
 - Easy for humans to read/write and for machines to parse/generate
 - Utilizing key-value pairs and arrays

2.4.4.2 ***Some Other Web Services Standards***

- HTTP (1.1/2): The foundation for web communication

- OpenAPI Specification (OAS): The industry standard for defining and describing RESTful APIs
- WS-Security: Provides enhancements for secure SOAP messaging.
- WS-Addressing: Defines transport-neutral mechanisms for addressing messages.
- WS-ReliableMessaging: Ensures messages are delivered in distributed systems.
- MTOM: Optimizes transmission of SOAP message

2.5 Simple Object Access Protocol (SOAP)

- SOAP plays the role of the messaging protocol for exchanging information between the service provider and the service requester
- Consists of: SOAP Envelope, SOAP Transport, SOAP Encoding, and SOAP RPC conventions

2.5.1 Components of a SOAP message

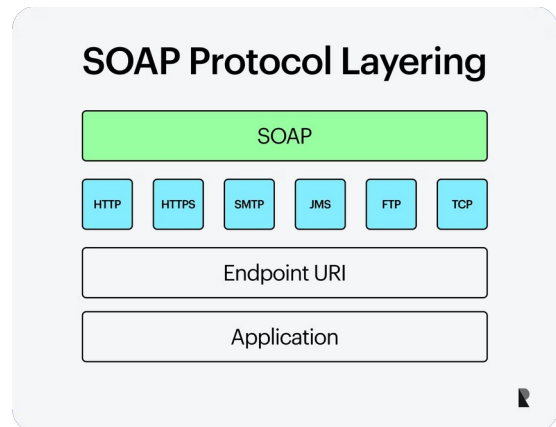
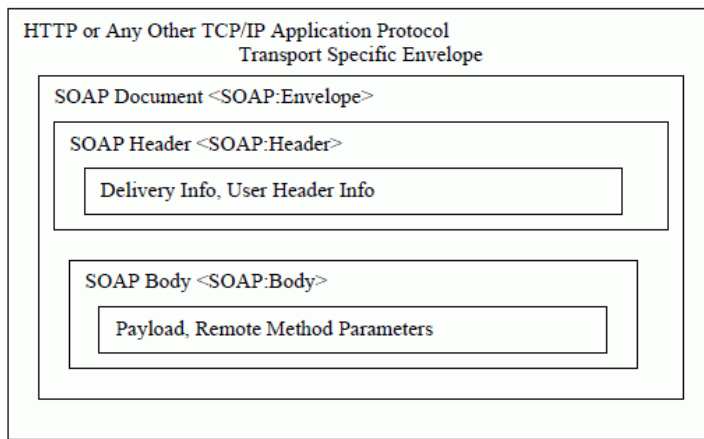
A SOAP message

- Is an ordinary XML document containing the following elements
 - An Envelope (the root element) that identifies the XML document as a SOAP message, identifying the contents and the envelope's processing information
 - An optional Header element that contains header information
 - A required Body element that contains call and response information
 - Optional body elements
 - Elements called Faults can be used to describe exceptional situations
 - Attachments in MIME encoding for exchanging binary data



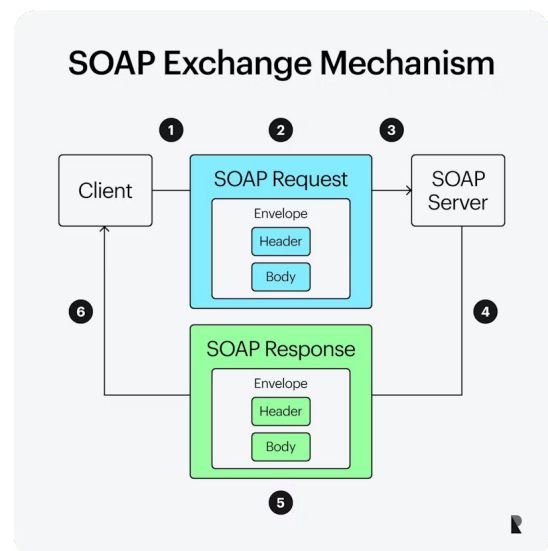
2.5.2 SOAP Transport

- Defines the bindings for the underlying transport protocols such as HTTP and SMTP



Steps

1. Client creates a SOAP request message and sends it to the SOAP server over a network using one of several protocols
2. The SOAP request message contains a SOAP envelope, which defines the structure of the message, including header information and body
3. The server receives the SOAP request message and performs various operations to validate it, retrieve data from a database, or execute business logic
4. The server creates and sends a SOAP response message back to the client using the same communication protocol used for sending requests.
5. The SOAP response contains a SOAP envelope, which defines the structure of the message as well as the payload, (i.e., the message being exchanged between the client and the server)
6. The client receives and processes the SOAP response message



2.5.3 SOAP Message Examples

```

POST /StockQuote HTTP/1.1
Host: www.acmeretailer.com
Content-Type: text/xml; charset="utf-8"
Content-Length: 1000
SOAPAction: "getBookPrice"
<SOAP-ENV:Envelope
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3c.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3c.org/2001/XMLSchema"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<m:getBookPrice
xmlns:m="http://www.wiley.com/jws.book.pricelist">
<bookname xsi:type='xsd:string'>
Developing Java Web services</bookname>
</m:getBookPrice>
/soap:Body>
</SOAP-ENV:Envelope><

```

```

<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  soap:encodingStyle="http://soap.org/soap/encoding/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-inst">
  <soap:Header>
    <!-- Transactions, priorities, etc. -->
  </soap:Header>
  <soap:Body>
    <!-- Some content -->
  </soap:Body>
</soap:Envelope>

```

2.6 SOAP vs REST

- These provide the 2 main approaches to building web services
 - SOAP is a strict, standards-based protocol
 - REST is a flexible architectural style
- Quick Comparison Table

Feature	SOAP (Protocol)	REST (Architectural Style)
Data Format	XML only	Flexible (JSON, XML, HTML, Text)
Transport	HTTP, SMTP, JMS, TCP	Exclusively HTTP/HTTPS
State	Can be stateful or stateless	Inherently stateless
Security	Built-in WS-Security (message-level)	Transport-level (HTTPS, OAuth 2.0, JWT)
Performance	Slower (heavier XML payloads)	Faster (lightweight JSON, caching support)
Contract	Required (WSDL file)	Optional (OpenAPI/Swagger)
Caching	Responses are not cacheable.	Built-in HTTP caching for improved speed.

- Key Differences

- Design Philosophy
 - SOAP is operation-based, focusing on calling specific actions like `GetUserInfo`
 - REST is resource-based, where you interact with "resources" via standard URLs and HTTP methods like `GET /users/123`
- Security and Compliance
 - SOAP is preferred for high-security sectors (banking, healthcare) because it supports WS-Security for end-to-end message encryption and ACID compliance for reliable database transactions
- Efficiency
 - REST is generally faster and uses less bandwidth because it uses JSON, which is more compact than the verbose XML "envelopes" required by SOAP
- Error Handling
 - SOAP has a standardized Fault element in its XML structure for detailed error reporting
 - REST relies on standard HTTP status codes (like 404 or 500) and custom response bodies.
- When to Use Each
 - Choose REST for: Modern web and mobile applications, public APIs, and microservices where speed and scalability are the top priorities
 - Choose SOAP for: Legacy system integrations, enterprise-level internal services, and mission-critical financial or healthcare transactions that require strict formal contracts and advanced security
- Sample code to illustrate the difference between REST and SOAP approaches
- The problem: Requesting user information for "User 123"

REST

Request	Response
http GET /users/123 HTTP/1.1 Host: api.example.com Accept: application/json	json { "id": 123, "name": "Jane Doe", "email": "jane@example.com" }

SOAP

Request	Response
xml POST /UserService HTTP/1.1 Host: api.example.com Content-Type: text/xml; charset=utf-8 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope" /> <soap:Body> <GetUserRequest xmlns="http://example.com"> <UserId>123</UserId> </GetUserRequest> </soap:Body> </soap:Envelope>	xml <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope" /> <soap:Body> <GetUserResponse xmlns="http://example.com"> <User> <Id>123</Id> <Name>Jane Doe</Name> <Email>jane@example.com</Email> </User> </GetUserResponse> </soap:Body> </soap:Envelope>

Note: SOAP requires a formal "Envelope" and is almost always sent via a POST request, even if you are just retrieving data. The XML structure is much more verbose.

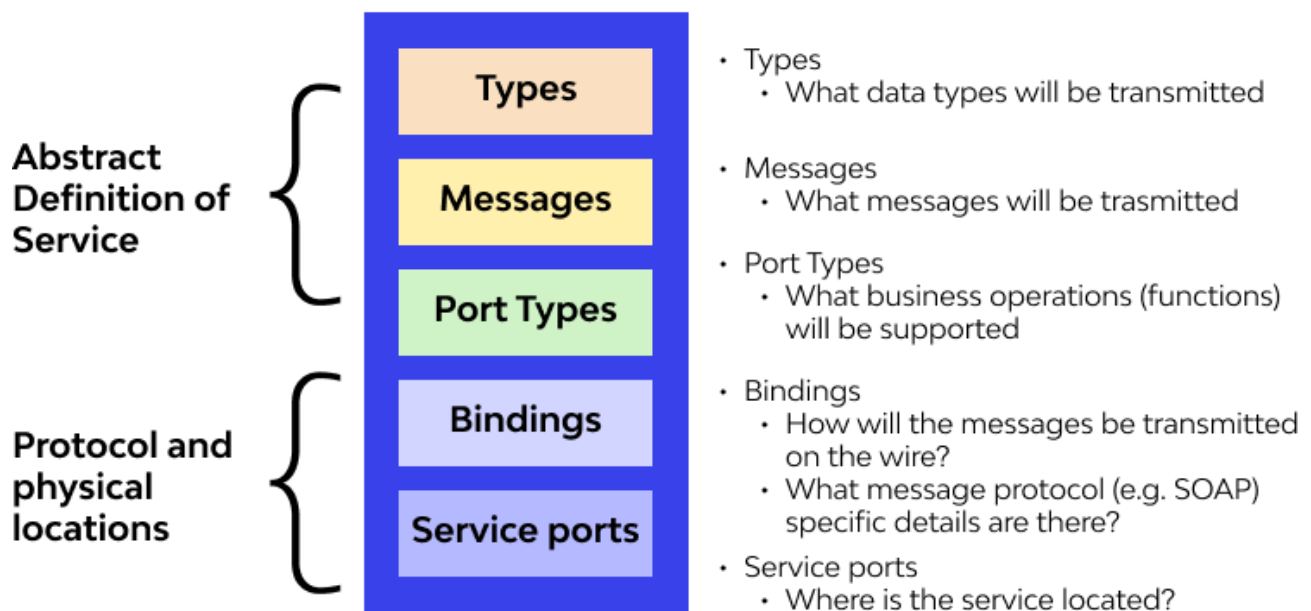
2.7 Web Services Description Language (WSDL)

WSDL (Web Services Description Language) is an XML-based language used to describe the functionality, location, and communication protocols of a web service. It acts as a formal "interface contract" between a client and a server, predominantly for

SOAP-based services.

- An XML schema-based specification used to describe the functionality, location, and communication protocols of a web service
- Acts as a formal "interface contract" between a client and a server, predominantly for SOAP-based services
 - Defines the available operations and the protocols required to access them
 - The "how," "what," and "where" of a web service

2.7.1.1 WSDL Document Structure



- A WSDL document is logically divided into two parts: abstract definitions (what the service does) and concrete bindings (how and where to access it)
- Organized into several key elements, typically nested within a root `<definitions>` element
 - `<definitions>`
 - The root element that encapsulates the entire service description and declares the necessary namespaces
 - `<types>`
 - Acts as a container for data type definitions
 - It primarily uses XML Schema (XSD) to define complex data structures exchanged between the client and server
 - `<message>`
 - Defines the data being communicated
 - Consists of one or more "parts," which can be compared to parameters in a function call
 - `<portType>` (WSDL 1.1) / `<interface>` (WSDL 2.0)
 - The most critical element
 - A logical grouping of operations
 - Describes the abstract operations (methods) provided by the service and the messages involved in those operations, i.e., which messages are used for input, output, or faults
 - `<binding>`
 - Links the abstract `portType` to a concrete protocol (e.g., SOAP over HTTP) and defines the message format (e.g., RPC or Document style)
 - `<service>`
 - Groups related endpoints (ports) together

- Contains the <port> or <endpoint> element, which specifies the actual network address (URL) of the service
- <port> (WSDL 1.1) / <endpoint> (WSDL 2.0)
 - Defines a single communication endpoint by associating a physical address (URL) with a specific binding

WSDL Operational Patterns

WSDL defines four transmission primitives to describe how messages flow between the client and the service

- One-way: The service receives a message but sends no response.
- Request-Response: The service receives a request and returns a response (most common).
- Solicit-Response: The service sends a request and waits for a response.
- Notification: The service sends a message without waiting for a response

WSDL Development Approaches

Developers typically use two methods to work with WSDL files

- Top-Down (Contract-First): Starting with a WSDL file and using tools like `wsimport` or `WSDL2Java` to generate the code scaffolding.
- Bottom-Up (Code-First): Starting with existing code (e.g., Java classes) and automatically generating the WSDL file using tools like `wsgen` or `Java2WSDL`.

WSDL Version Comparison

Feature	WSDL 1.1 (2001)	WSDL 2.0 (2007)
Status	W3C Note (Widely used)	W3C Recommendation
Root Element	<definitions>	<description>
Interface Element	<portType>	<interface>
Endpoint Element	<port>	<endpoint>
Inheritance	Not supported	Supports interface inheritance (<code>extends</code>)

2.8 Implementing Web Services

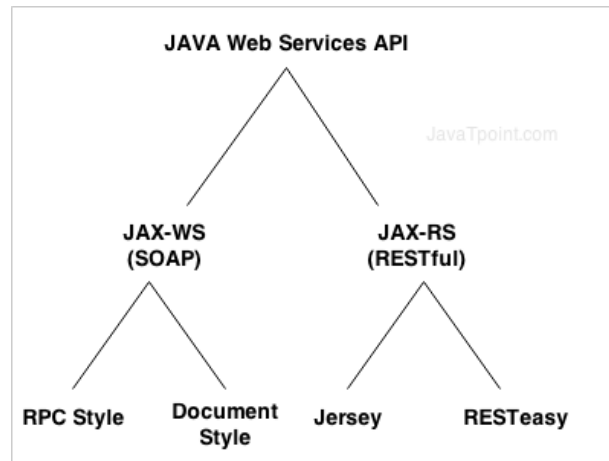
- Process involves creating software components that communicate over a network using standardized protocols like HTTP and XML to ensure interoperability between different platforms and languages
- Requirements and Design
 - Define Functionality: Identify the business logic or data that needs to be exposed (e.g., retrieving customer data or processing transactions)
 - Select Architecture: Choose between RESTful (typically using JSON over HTTP) or SOAP-based (XML-based messaging) services

- Establish a Contract: For SOAP, this involves creating a WSDL file to define operations and message formats
- Development (Coding)
 - Write the Logic: Develop the back-end components (e.g., Java classes, EJBs, or .NET scripts) that perform the actual processing
 - Expose Operations: Annotate or configure specific methods to be accessible as web service "operations"
 - Handle Data Types: Ensure complex data types are mapped correctly between the programming language and the exchange format (XML/JSON)
- Deployment
 - Configure the Server: Set up a host environment, such as Oracle WebLogic, GlassFish, etc.
 - Publish the Service: Deploy the compiled code to the application server so it can receive network requests
- Testing and Validation
 - Verify Endpoints: Use built-in IDE testers or tools like SOAPUI or Postman to send test requests and verify the XML/JSON responses
 - Check Security: Ensure that authentication (e.g., Basic Auth, SSL) and authorization policies are working correctly
- Discovery and Consumption
 - Register the Service: Optionally publish the service details in a registry like UDDI so potential clients can find it
 - Create Client Proxies: Consumers use the WSDL or API definition to generate "proxies" that allow their applications to call the service as if it were a local function.

2.9 Java Web Services Development

In this course, we shall use Java to build web services.

2.9.1 The Java Web Services API



Two main API's defined by Java for developing web service applications

- JAX-WS (Java API for XML-Web Services)
 - Commonly shortened to JWS for Java Web Services
 - For developing SOAP web services (although there is also some support for RESTful web services)
 - All of the libraries required to compile, execute, and consume web services are available in core Java 6 (and above), which supports JAX-WS
 - Two ways to write JAX-WS application code
 - By RPC style
 - By Document style
- JAX-RS (Java API for XML-RESTful Web Services)
 - For RESTful web services
 - 2 main implementation for creating JAX-RS application
 - Jersey and RESTeasy
- Modern Framework: Spring Boot
 - While JAX-RS and JAX-WS are official standards, most modern Java developers use Spring Boot to build Web APIs
 - We do not treat Spring Boot in this course